Assignment 3 (Python): Reverse a String

CS 101 Programming Principles · Fall 2026

LANGUAGE	FILE TO WRITE	DUE
Python 3.10+ (standard library only)	`reverse.py`	**Nov 20, 2026, 11:59 PM**
POINTS	YOU SUBMIT	TIME
20 (demo course)	`reverse.py` **+ your** `.codelapse` **recording**	**About 45–60 minutes, then up to 1 hour for 4 short questions**

1. Overview

You will write a small Python program with two functions: one that returns a string reversed, and one that counts its vowels. Python could reverse a string for you in one line, so this assignment asks you **not** to use that shortcut and to build the reversed string yourself with a loop. That way you practise the ideas the shortcut hides: indexing, loops that count down, and the fact that Python strings cannot be changed in place.

You will write it in VS Code while the **CodeLapse** extension records how you work. When you submit, you will answer four short questions about your own work and your own code.

What you will practise

- Indexing into a string (`text[i]`), `len` , and loops with `for` and `range` .
- Why strings are **immutable**, and how to build a new string (or a list of characters) instead.
- Writing functions that **return** a value, and running code only when the file is run as a program.
- Testing your own code on ordinary and unusual inputs.

2. What to build

Write everything in one file, `reverse.py` . Use only plain Python (no imports are needed).

`def reverse(text: str) -> str`

- **Returns a new string** with the characters of `text` in reverse order. It must not print anything.
- Must build the result **with a loop** of your own: for example by going through the string from the end, or by swapping characters in a list.
- Must work for any string, including the empty string `""` and a string of one character.

`def count_vowels(text: str) -> int`

- Returns how many vowels the string contains. The vowels are `a e i o u` , in **either case** (`A E I O U` count too).
- `y` is **not** a vowel for this assignment. Spaces, digits and punctuation are not vowels.

Running it as a program

- At the bottom, under `if __name__ == "__main__":` , put a test word in a variable (for example `"codelapse"`), call both functions on it, and print the reversed word and the vowel count.

> **Not allowed in `reverse`**
>
> Slicing that reverses (`text[::-1]` or any slice with a negative step), `reversed()`, `list.reverse()`, `sorted()` tricks, recursion instead of a loop, or any import or library that reverses for you. Allowed: `for` and `while` loops, `range`, `len`, indexing like `text[i]`, lists, `append`, and `"".join(...)` on a list you built yourself.

3. Examples

Every row was checked by running a correct solution. The quotes only show where the string starts and ends.

INPUT	`REVERSE(...)` RETURNS	`COUNT_VOWELS`	WHY IT IS A USEFUL TEST
`"codelapse"`	`"espaledoc"`	4	The normal case (odd length: 9)
`""`	`""`	0	Empty string: the loop must not fail
`"a"`	`"a"`	1	One character: stays the same
`"ab"`	`"ba"`	1	Shortest even length
`"abc"`	`"cba"`	1	Short odd length
`"racecar"`	`"racecar"`	3	A palindrome reads the same reversed
`"a b"`	`"b a"`	1	Spaces are characters too
`"Hello, World!"`	`"!dlroW ,olleH"`	3	Punctuation, a space, and capitals
`"Programming"`	`"gnimmargorP"`	3	A capital letter moves to the end
`"AEIOU aeiou"`	`"uoiea UOIEA"`	10	Upper- and lower-case vowels both count
`"rhythm"`	`"mhtyhr"`	0	`y` is not a vowel

Example run

```
$ python3 reverse.py
Original: codelapse
Reversed: espaledoc
Vowels: 4
```

The exact wording of the output is up to you, but it must show the reversed word and the vowel count. Because `reverse` returns a new string, the original variable still holds the original word afterwards.

4. A plan that works

1. Create the folder, open it in VS Code and start CodeLapse (section 8). Create `reverse.py`.
2. Write the `if __name__ == "__main__":` block so it only prints your test word. Run it, so you know your setup works.
3. Write `reverse`. Print `reverse(word)` and run.
4. Try the examples above, by changing the test word or by printing several results.
5. Write `count_vowels`. Print its result and check it against the table.
6. Add a few `assert` checks (section 6), run once more, and save the file.

5. Hints

Read one hint at a time and try it yourself before moving on to the next. They start general and get more specific.

1 GETTING STARTED

A string is a sequence: `text[0]` is the first character and `text[len(text) - 1]` is the last. Indexing past the end raises `IndexError`.

2 STRINGS CAN'T BE CHANGED

Python strings are **immutable**: `text[0] = "x"` is an error. So `reverse` cannot change `text`; it has to **build a new string** and return it. Two common ways: start from an empty string and add characters to it, or turn the text into a list of characters, rearrange the list, then join it back with `"".join(...)`.

3 WHICH ORDER TO VISIT THE CHARACTERS

If you visit the characters from the **last to the first**, adding each one to the end of your result gives the reversed string. Alternatively, visit them first to last and put each new character at the **front** of your result. Try both on paper with `"abc"`.

4 COUNTING DOWN WITH RANGE

`range(start, stop, step)` can count down when `step` is negative, for example `range(5, 0, -1)` gives 5, 4, 3, 2, 1. The `stop` value is never included. Work out which `start` and `stop` make the last index you visit be `0`, and check it with `"ab"` and `""`.

5 THE LIST-AND-SWAP WAY

Like in C: put the characters in a list, keep one index at the start and one at the end, swap, and move both towards the middle. Stop at the middle; going further swaps everything back.

6 COUNTING VOWELS

Go through the characters and count the ones that are vowels. `ch in "aeiou"` is `True` when `ch` is one of those letters (a set such as `{"a", "e", "i", "o", "u"}` works too). To include capitals, compare a lower-case version of each character: `ch.lower()` (or lower-case the whole text once).

7 RETURN, DON'T PRINT

Both functions should `return` their result. The printing belongs in the `if __name__ == "__main__":` block. A function that prints and returns `None` looks right on screen but fails as soon as anything else uses it.

6. Test your program

- Run `python3 reverse.py` in the VS Code terminal after every change (on Windows the command may be `py reverse.py` or `python reverse.py`).
- Compare with the table, and try the edge cases by changing the test word.
- Let Python check for you with `assert` lines in the `__main__` block. An `assert` stays silent when the condition is true and stops with an `AssertionError` when it is false:

```
assert reverse("ab") == "ba"
assert count_vowels("rhythm") == 0
```

Self-check before you submit

- [ ] Runs with `python3 reverse.py` without errors.
- [ ] `reverse` uses a loop of your own: no `[::-1]`, no `reversed()`, no `.reverse()`.
- [ ] Correct for `""`, `"a"`, `"ab"`, `"abc"` and `"Hello, World!"`.
- [ ] `count_vowels` counts capital vowels and does not count `y`.
- [ ] Both functions **return** their result, with the names and parameters exactly as specified.
- [ ] The printing happens under `if __name__ == "__main__":`.
- [ ] You can explain, in your own words, how your loop visits the characters and why it starts and stops where it does.

7. Common mistakes

MISTAKE	WHAT HAPPENS, AND THE FIX
Using `[::-1]` or `reversed()`	It works, but it is not allowed here: the point is to write the loop yourself.
Off by one in `range`	The first character goes missing, or an `IndexError` appears. Remember `stop` is excluded; test with `"ab"`.
Trying to change the string	`text[i] = ...` raises `TypeError`. Build a new string or use a list.
Printing instead of returning	The function returns `None`. Use `return`, and print in the `__main__` block.
Forgetting capital vowels, or counting `y`	`"AEIOU aeiou"` must give 10 and `"rhythm"` must give 0.
Indentation errors	Python reports `IndentationError`. Indent every block by the same amount (VS Code uses 4 spaces).
Running a different file	Make sure the terminal is in your assignment folder and you run `reverse.py`.

8. Working with CodeLapse

CodeLapse records how your program grows, so the questions can be about *your* work. Set it up before you start writing. The full tester guide (installation and troubleshooting) is in `docs/TESTER_GUIDE.md` of the CodeLapse repository.

1. **Check your tools.** VS Code 1.93 or newer, and Python 3.10 or newer: `python3 --version` in a terminal (on Windows `py --version`) should print a version.
2. **Install CodeLapse.** Download `codelapse-0.1.0.vsix` from
   https://github.com/Shanthan2307/Codelapse_ext/releases/tag/v0.1.0-pilot
   In VS Code: **Extensions** → `···` → **Install from VSIX…** and pick the file. A short walkthrough opens.
3. **Choose your folder.** Create an empty folder (for example `reverse-assignment`), open it with **File** → **Open Folder…**, then click **CodeLapse: Choose folder** in the status bar and pick it. The status bar now shows the folder name with a record icon. Only this folder is recorded.

4. **Write and run in VS Code.** Create `reverse.py`, and run it in the VS Code terminal (**Terminal → New Terminal**, bash, zsh or PowerShell) with `python3 reverse.py`. Runs in a separate terminal app are not recorded. Errors along the way are fine and normal.
5. **Export.** Open the Command Palette (`Cmd+Shift+P` / `Ctrl+Shift+P`) and run **CodeLapse: Export Assignment Bundle (.codelapse)**. Save the `.codelapse` file.
6. **Submit.** Click **Open Submission Page** in the message that appears (or run **CodeLapse: Open Submission Page**). The first time, VS Code asks for the address your instructor gives you: make sure it is the address of the **Python** assignment. On the page, pick your name, add the `.codelapse` file and `reverse.py`, and click **Submit Assignment**.

9. The four questions

Right after you submit, the page shows **four short questions about your own work**: two about things you did while working (for example an error you fixed, or something you rewrote), and two about specific lines of your code. Every student gets different questions.

- **You have one hour from submitting** to answer, and you can submit your answers **once**. A timer shows the time left; your draft is kept if you refresh the page.
- Write a few sentences each, **in your own words**. Spelling and grammar do not matter.
- Explain **why**, not just what: why the loop starts and stops where it does, why you chose that approach, what an error message told you.
- Be specific: mention the actual line, variable, error or change. "It works" says much less than "the loop starts at the last index because …".
- If you are not sure, say what you think and why. An honest partial answer is better than a vague one.

10. Working honestly, and what is recorded

Write the code yourself, and please turn off AI coding assistants (such as GitHub Copilot) for this assignment. Mistakes, errors and rewrites are completely normal; they are part of how programs get written.

CodeLapse records only the folder you choose: what you type, when you switch files, and the programs you run in the VS Code terminal with their output. It does not record other folders, websites, other apps or your file paths and username. Nothing leaves your computer until you export and upload it yourself.

Your instructor sees your code, your answers, a summary of your recording (how long you worked, how much was typed or added in larger pieces such as pastes, how your runs went), and a short review of how well your answers explain your code. None of this is a grade or an accusation: it describes how the work happened, and your instructor decides what it means. If you are unsure whether something is allowed, ask your instructor.

11. If something goes wrong

WHAT YOU SEE	WHAT TO DO
`python3: command not found`	Install Python 3 from python.org. On Windows, try `py` or `python` instead of `python3`.
Status bar says **CodeLapse: Choose folder** or **Not recording**	Click it and choose your assignment folder.
Runs don't seem to be recorded	Run your program in the VS Code terminal (bash, zsh or PowerShell), not a separate Terminal app.
"Choose who you are before submitting."	Pick your name at the top of the form.
"Attach your .codelapse recording."	Add the `.codelapse` file you exported.
"The recording has no sessions in it."	Nothing was recorded in that folder. Check you chose the right folder.
The page says a file is not allowed	Upload only `reverse.py`, not other files.
"Your questions could not be prepared."	Nothing to do: your submission was received and your instructor has been told.

Anything else: ask your instructor. Good luck, and have fun with it.